

Computer Vision and Deep Learning

Week 3 Manual

Introduction to PyTorch and Convolutional Neural Networks

Learning Objectives

By the end of this week, you will:

- Understand PyTorch basics and tensor operations
- Build your first Convolutional Neural Network using PyTorch
- Train CNNs on standard image datasets
- Understand the complete deep learning pipeline
- Compare manual implementation with framework implementation

Prerequisites:

- Completion of Week 1 & 2 assignments
- Understanding of convolution and image filtering
- Basic neural network concepts from Week 1
- Python programming with NumPy

July 8, 2025

Contents

1	Assignment 1: PyTorch Fundamentals and CNN Building	2
1.1	Learning Material	2
1.1.1	Introduction to PyTorch	2
1.1.2	Core PyTorch Concepts	2
1.1.3	CNN Components in PyTorch	2
1.2	Demo Module	2
1.2.1	PyTorch Basics and Simple CNN	2
1.3	Task Modules	5
1.3.1	Task 1.1: PyTorch Tensor Practice (Monday)	5
1.3.2	Task 1.2: Build Your First CNN (Tuesday)	5
1.4	Assessment Criteria for Assignment 1	6
2	Assignment 2: CNN Training and Evaluation	7
2.1	Learning Material	7
2.1.1	Deep Learning Training Pipeline	7
2.1.2	Training Loop Components	7
2.1.3	Monitoring and Evaluation	7
2.2	Demo Module	7
2.2.1	Complete Training Pipeline	7
2.3	Task Modules	11
2.3.1	Task 2.1: Implement Training Pipeline (Wednesday)	11
2.3.2	Task 2.2: Model Evaluation and Analysis (Thursday)	12
2.4	Assessment Criteria for Assignment 2	12
3	Weekly Assessment (Friday)	12
4	Resources and Tools	12
4.1	Required Software	12
4.2	Recommended Datasets	12
4.3	Additional Reading	13
5	Next Week Preview	13

1 Assignment 1: PyTorch Fundamentals and CNN Building

Duration: Monday - Tuesday — **Assessment:** Tuesday End of Day

1.1 Learning Material

1.1.1 Introduction to PyTorch

PyTorch is a popular deep learning framework that makes building and training neural networks easier.

1.1.2 Core PyTorch Concepts

Tensors: Multi-dimensional arrays similar to NumPy arrays but with additional capabilities:

- Can run on GPU for faster computation
- Support automatic differentiation for gradient computation
- Integrated with neural network operations

Autograd: PyTorch's automatic differentiation system that:

- Tracks operations on tensors
- Computes gradients automatically
- Enables backpropagation without manual implementation

nn.Module: Base class for all neural network layers and models:

- Manages parameters automatically
- Provides methods for training and evaluation modes
- Handles GPU movement and state saving

1.1.3 CNN Components in PyTorch

- **nn.Conv2d:** Convolutional layers for feature extraction
- **nn.MaxPool2d:** Pooling layers for dimension reduction
- **nn.ReLU:** Activation functions for non-linearity
- **nn.Linear:** Fully connected layers for final classification
- **nn.CrossEntropyLoss:** Loss function for classification tasks
- **torch.optim:** Optimizers for updating network weights

1.2 Demo Module

1.2.1 PyTorch Basics and Simple CNN

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torch.nn.functional as F
5 import torchvision
6 import torchvision.transforms as transforms
7 import matplotlib.pyplot as plt
8 import numpy as np
9
10 # Check PyTorch installation and GPU availability
11 print(f"PyTorch version: {torch.__version__}")
12 print(f"CUDA available: {torch.cuda.is_available()}")
13 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
14 print(f"Using device: {device}")
```

```

15
16 # 1. Tensor Operations - Building blocks of PyTorch
17 print("\n=== Tensor Operations ===")
18
19 # Creating tensors
20 x = torch.tensor([[1, 2], [3, 4]], dtype=torch.float32)
21 print(f"Tensor x:\n{x}")
22
23 # Basic operations
24 y = torch.tensor([[5, 6], [7, 8]], dtype=torch.float32)
25 z = x + y # Element-wise addition
26 print(f"x + y:\n{z}")
27
28 # Reshape operations
29 reshaped = x.view(-1) # Flatten
30 print(f"Flattened x: {reshaped}")
31
32 # 2. Autograd demonstration
33 print("\n=== Automatic Differentiation ===")
34
35 # Create tensor with gradient tracking
36 x = torch.tensor([2.0], requires_grad=True)
37 y = x**2 + 3*x + 1
38
39 # Compute gradient
40 y.backward()
41 print(f"x = {x.item()}")
42 print(f"y = x^2 + 3x + 1 = {y.item()}")
43 print(f"dy/dx = 2x + 3 = {x.grad.item()}")
44
45 # 3. Simple CNN Architecture
46 class SimpleCNN(nn.Module):
47     def __init__(self, num_classes=10):
48         super(SimpleCNN, self).__init__()
49
50         # Convolutional layers
51         self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1) # 3 input
52         # channels, 16 output
53         self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1) # 16 input,
54         # 32 output
55
56         # Pooling layer
57         self.pool = nn.MaxPool2d(2, 2) # 2x2 pooling
58
59         # Fully connected layers
60         self.fc1 = nn.Linear(32 * 8 * 8, 128) # Depends on input image size
61         self.fc2 = nn.Linear(128, num_classes)
62
63         # Dropout for regularization
64         self.dropout = nn.Dropout(0.5)
65
66     def forward(self, x):
67         # First conv block: conv -> relu -> pool
68         x = self.pool(F.relu(self.conv1(x)))
69
70         # Second conv block: conv -> relu -> pool
71         x = self.pool(F.relu(self.conv2(x)))
72
73         # Flatten for fully connected layers
74         x = x.view(-1, 32 * 8 * 8)
75
76         # Fully connected layers with dropout
77         x = F.relu(self.fc1(x))

```

```

76         x = self.dropout(x)
77         x = self.fc2(x)
78
79         return x
80
81 # 4. Data Loading and Preprocessing
82 transform = transforms.Compose([
83     transforms.ToTensor(),
84     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
85 ])
86
87 # Load CIFAR-10 dataset (small subset for demo)
88 trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
89                                         download=True, transform=transform)
90 trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
91                                           shuffle=True, num_workers=2)
92
93 testset = torchvision.datasets.CIFAR10(root='./data', train=False,
94                                         download=True, transform=transform)
95 testloader = torch.utils.data.DataLoader(testset, batch_size=4,
96                                          shuffle=False, num_workers=2)
97
98 # CIFAR-10 class names
99 classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship',
100            'truck']
101
102 # 5. Visualize some training data
103 def show_images(dataloader):
104     dataiter = iter(dataloader)
105     images, labels = next(dataiter)
106
107     fig, axes = plt.subplots(1, 4, figsize=(12, 3))
108     for i in range(4):
109         # Convert tensor to numpy and denormalize
110         img = images[i] / 2 + 0.5
111         img = img.permute(1, 2, 0)
112
113         axes[i].imshow(img)
114         axes[i].set_title(f'Class: {classes[labels[i]]}')
115         axes[i].axis('off')
116     plt.tight_layout()
117     plt.show()
118
119 print("\n=== Sample Training Images ===")
120 show_images(trainloader)
121
122 # 6. Initialize model, loss function, and optimizer
123 model = SimpleCNN(num_classes=10).to(device)
124 criterion = nn.CrossEntropyLoss()
125 optimizer = optim.Adam(model.parameters(), lr=0.001)
126
127 # Print model architecture
128 print("\n=== Model Architecture ===")
129 print(model)
130
131 # Count parameters
132 total_params = sum(p.numel() for p in model.parameters())
133 print(f"\nTotal parameters: {total_params:,}")
134
135 # 7. Simple training demonstration (just a few batches)
136 print("\n=== Training Demonstration ===")
137 model.train()

```

```

138 for i, (inputs, labels) in enumerate(trainloader):
139     if i >= 5: # Just train on 5 batches for demonstration
140         break
141
142     inputs, labels = inputs.to(device), labels.to(device)
143
144     # Zero gradients
145     optimizer.zero_grad()
146
147     # Forward pass
148     outputs = model(inputs)
149     loss = criterion(outputs, labels)
150
151     # Backward pass and optimization
152     loss.backward()
153     optimizer.step()
154
155     print(f'Batch {i+1}, Loss: {loss.item():.4f}')
156
157 print("Demo training complete!")

```

Listing 1: PyTorch Fundamentals and CNN Implementation

Expected Output: Working PyTorch installation heck, tensor operations, gradient computation, CNN architecture, and training demonstration.

1.3 Task Modules

1.3.1 Task 1.1: PyTorch Tensor Practice (Monday)

1. Explore tensor operations:
 - Create tensors with different shapes and data types
 - Practice reshaping, slicing, and indexing operations
 - Perform mathematical operations (addition, multiplication, matrix operations)
 - Move tensors between CPU and GPU (if available)
2. Understand automatic differentiation:
 - Create simple functions and compute their gradients
 - Practice with multi-variable functions
 - Understand when gradients are computed vs when they're not
3. Compare PyTorch with NumPy:
 - Implement the same operations in both PyTorch and NumPy
 - Note similarities and differences in syntax
 - Convert between PyTorch tensors and NumPy arrays

Deliverable: Python script demonstrating tensor operations, gradient computation, and PyTorch-NumPy comparison

1.3.2 Task 1.2: Build Your First CNN (Tuesday)

1. Design a simple CNN architecture:
 - Create a CNN with 2 convolutional layers and 2 fully connected layers
 - Use appropriate activation functions and pooling
 - Calculate the output dimensions at each layer

2. Implement the forward pass:
 - Define the forward method correctly
 - Test with random input tensors
 - Verify output shapes are as expected
3. Explore model components:
 - Print model parameters and their shapes
 - Count total number of parameters
 - Visualize model architecture using torchsummary (optional)

Deliverable: CNN implementation with architecture details and parameter analysis

1.4 Assessment Criteria for Assignment 1

- **PyTorch Understanding (40%):** Correct use of tensors, operations, and autograd
- **CNN Implementation (35%):** Proper CNN architecture with correct forward pass
- **Code Quality (15%):** Clean, well-commented, and organized code
- **Analysis (10%):** Good understanding of model structure and parameters

2 Assignment 2: CNN Training and Evaluation

Duration: Wednesday - Thursday — **Assessment:** Thursday End of Day

2.1 Learning Material

2.1.1 Deep Learning Training Pipeline

A complete training pipeline consists of several key components:

1. **Data preparation:** Loading, preprocessing, and augmentation
2. **Model initialization:** Setting up the network architecture
3. **Training loop:** Forward pass, loss computation, backpropagation
4. **Validation:** Monitoring performance on unseen data
5. **Evaluation:** Final testing and performance analysis

2.1.2 Training Loop Components

Forward Pass:

- Input data flows through the network
- Each layer transforms the data
- Final layer produces predictions

Loss Computation:

- Compare predictions with ground truth labels
- Common losses: CrossEntropyLoss for classification, MSELoss for regression
- Loss quantifies how wrong the predictions are

Backward Pass:

- Compute gradients of loss with respect to parameters
- PyTorch autograd handles this automatically
- Gradients indicate how to update parameters to reduce loss

Parameter Update:

- Optimizer uses gradients to update network parameters
- Common optimizers: SGD, Adam, RMSprop
- Learning rate controls the step size of updates

2.1.3 Monitoring and Evaluation

- **Training metrics:** Track loss and accuracy during training
- **Validation metrics:** Monitor performance on held-out data
- **Overfitting detection:** When training accuracy increases but validation decreases
- **Learning curves:** Plots showing training progress over time

2.2 Demo Module

2.2.1 Complete Training Pipeline

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torch.nn.functional as F
5 import torchvision
6 import torchvision.transforms as transforms
7 import matplotlib.pyplot as plt
8 import numpy as np
9 import time
10
11 # Device configuration
12 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
13
14 # Data preprocessing with augmentation for training
15 transform_train = transforms.Compose([
16     transforms.RandomHorizontalFlip(p=0.5),
17     transforms.ToTensor(),
18     transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010))
19 ])
20
21 transform_test = transforms.Compose([
22     transforms.ToTensor(),
23     transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010))
24 ])
25
26 # Load datasets
27 trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
28                                         download=True, transform=transform_train
29 )
29 trainloader = torch.utils.data.DataLoader(trainset, batch_size=32,
30                                           shuffle=True, num_workers=2)
31
32 testset = torchvision.datasets.CIFAR10(root='./data', train=False,
33                                         download=True, transform=transform_test)
34 testloader = torch.utils.data.DataLoader(testset, batch_size=32,
35                                         shuffle=False, num_workers=2)
36
37 classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship',
38            'truck']
39
40 # Improved CNN architecture
41 class ImprovedCNN(nn.Module):
42     def __init__(self, num_classes=10):
43         super(ImprovedCNN, self).__init__()
44
45         self.conv1 = nn.Conv2d(3, 32, 3, padding=1)
46         self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
47         self.conv3 = nn.Conv2d(64, 128, 3, padding=1)
48
49         self.pool = nn.MaxPool2d(2, 2)
50
51         self.fc1 = nn.Linear(128 * 4 * 4, 256)
52         self.fc2 = nn.Linear(256, num_classes)
53
54         self.dropout = nn.Dropout(0.5)
55
56     def forward(self, x):
57         x = self.pool(F.relu(self.conv1(x)))
58         x = self.pool(F.relu(self.conv2(x)))
59         x = self.pool(F.relu(self.conv3(x)))
60
61         x = x.view(-1, 128 * 4 * 4)
62         x = F.relu(self.fc1(x))

```

```

62         x = self.dropout(x)
63         x = self.fc2(x)
64
65         return x
66
67 # Initialize model
68 model = ImprovedCNN().to(device)
69 criterion = nn.CrossEntropyLoss()
70 optimizer = optim.Adam(model.parameters(), lr=0.001)
71
72 # Training function
73 def train_model(model, trainloader, criterion, optimizer, num_epochs=5):
74     model.train()
75     train_losses = []
76     train_accuracies = []
77
78     for epoch in range(num_epochs):
79         running_loss = 0.0
80         correct = 0
81         total = 0
82         start_time = time.time()
83
84         for i, (inputs, labels) in enumerate(trainloader):
85             inputs, labels = inputs.to(device), labels.to(device)
86
87             # Zero gradients
88             optimizer.zero_grad()
89
90             # Forward pass
91             outputs = model(inputs)
92             loss = criterion(outputs, labels)
93
94             # Backward pass
95             loss.backward()
96             optimizer.step()
97
98             # Statistics
99             running_loss += loss.item()
100             _, predicted = torch.max(outputs, 1)
101             total += labels.size(0)
102             correct += (predicted == labels).sum().item()
103
104             if i % 200 == 199:
105                 print(f'[Epoch {epoch+1}, Batch {i+1}] Loss: {running_loss
/200:.3f}')
106                 running_loss = 0.0
107
108             # Calculate epoch metrics
109             epoch_loss = running_loss / len(trainloader)
110             epoch_acc = 100 * correct / total
111             train_losses.append(epoch_loss)
112             train_accuracies.append(epoch_acc)
113
114             epoch_time = time.time() - start_time
115             print(f'Epoch {epoch+1}/{num_epochs} completed in {epoch_time:.2f}s')
116             print(f'Training Accuracy: {epoch_acc:.2f}%')
117             print('-' * 50)
118
119         return train_losses, train_accuracies
120
121 # Testing function
122 def test_model(model, testloader):
123     model.eval()

```

```

124     correct = 0
125     total = 0
126     class_correct = list(0. for i in range(10))
127     class_total = list(0. for i in range(10))
128
129     with torch.no_grad():
130         for inputs, labels in testloader:
131             inputs, labels = inputs.to(device), labels.to(device)
132             outputs = model(inputs)
133             _, predicted = torch.max(outputs, 1)
134             total += labels.size(0)
135             correct += (predicted == labels).sum().item()
136
137             # Per-class accuracy
138             c = (predicted == labels).squeeze()
139             for i in range(labels.size(0)):
140                 label = labels[i]
141                 class_correct[label] += c[i].item()
142                 class_total[label] += 1
143
144     overall_accuracy = 100 * correct / total
145     print(f'Overall Test Accuracy: {overall_accuracy:.2f}%')
146
147     # Print per-class accuracy
148     print('\nPer-class Accuracy:')
149     for i in range(10):
150         if class_total[i] > 0:
151             class_acc = 100 * class_correct[i] / class_total[i]
152             print(f'{classes[i]}: {class_acc:.2f}%')
153
154     return overall_accuracy
155
156 # Train the model
157 print("Starting training...")
158 train_losses, train_accs = train_model(model, trainloader, criterion, optimizer
159 , num_epochs=3)
160
161 # Test the model
162 print("\nEvaluating model...")
163 test_accuracy = test_model(model, testloader)
164
165 # Plot training curves
166 plt.figure(figsize=(12, 5))
167 plt.subplot(1, 2, 1)
168 plt.plot(train_losses)
169 plt.title('Training Loss')
170 plt.xlabel('Epoch')
171 plt.ylabel('Loss')
172
173 plt.subplot(1, 2, 2)
174 plt.plot(train_accs)
175 plt.title('Training Accuracy')
176 plt.xlabel('Epoch')
177 plt.ylabel('Accuracy (%)')
178 plt.tight_layout()
179 plt.show()
180
181 # Visualize predictions
182 def visualize_predictions(model, testloader, num_images=8):
183     model.eval()
184     dataiter = iter(testloader)
185     images, labels = next(dataiter)
186     images, labels = images.to(device), labels.to(device)

```

```

186
187     with torch.no_grad():
188         outputs = model(images)
189         _, predicted = torch.max(outputs, 1)
190
191     # Move back to CPU for visualization
192     images = images.cpu()
193     labels = labels.cpu()
194     predicted = predicted.cpu()
195
196     fig, axes = plt.subplots(2, 4, figsize=(15, 8))
197     for i in range(num_images):
198         ax = axes[i//4, i%4]
199         # Denormalize image
200         img = images[i]
201         img = img * torch.tensor([0.2023, 0.1994, 0.2010]).view(-1, 1, 1)
202         img = img + torch.tensor([0.4914, 0.4822, 0.4465]).view(-1, 1, 1)
203         img = torch.clamp(img, 0, 1)
204         img = img.permute(1, 2, 0)
205
206         ax.imshow(img)
207         ax.set_title(f'True: {classes[labels[i]]}\nPred: {classes[predicted[i]]}')
208         ax.axis('off')
209     plt.tight_layout()
210     plt.show()
211
212 visualize_predictions(model, testloader)

```

Listing 2: CNN Training and Evaluation Pipeline

Expected Output: Complete training process with loss/accuracy tracking, final test results, training curves, and prediction visualizations.

2.3 Task Modules

2.3.1 Task 2.1: Implement Training Pipeline (Wednesday)

1. Set up data loading for a standard dataset:
 - Choose either CIFAR-10 or Fashion-MNIST
 - Implement proper data preprocessing and normalization
 - Add basic data augmentation (horizontal flip, rotation)
 - Create train and test data loaders with appropriate batch sizes
2. Implement the complete training loop:
 - Forward pass, loss computation, backward pass, parameter update
 - Track training loss and accuracy for each epoch
 - Add progress reporting every few batches
 - Save model checkpoints periodically
3. Train your CNN from Assignment 1:
 - Train for at least 5 epochs
 - Monitor training progress and loss convergence
 - Experiment with different learning rates (0.01, 0.001, 0.0001)

Deliverable: Working training pipeline with progress tracking and learning rate experiments

2.3.2 Task 2.2: Model Evaluation and Analysis (Thursday)

1. Comprehensive model evaluation:
 - Implement testing function with overall and per-class accuracy
 - Calculate and display confusion matrix for test set
 - Identify which classes are easiest/hardest to classify
2. Visualization and analysis:
 - Plot training loss and accuracy curves
 - Visualize correct and incorrect predictions with images
 - Analyze common failure cases and misclassifications
3. Compare different approaches:
 - Train models with different architectures (more/fewer layers)
 - Compare different optimizers (SGD vs Adam)
 - Test with and without data augmentation
 - Document which approach works best and why

Deliverable: Complete evaluation report with visualizations, confusion matrix, and comparison of different approaches

2.4 Assessment Criteria for Assignment 2

- **Training Implementation (35%):** Correct training loop with proper data handling
- **Model Evaluation (30%):** Comprehensive testing and accuracy calculation
- **Analysis & Visualization (25%):** Good plots, confusion matrix, and insights
- **Experimental Comparison (10%):** Meaningful comparison of different approaches

3 Weekly Assessment (Friday)

Comprehensive evaluation covering both assignments

The weekly assessment will focus on understanding PyTorch fundamentals, CNN implementation, and the complete training/evaluation pipeline.

4 Resources and Tools

4.1 Required Software

- PyTorch (`pip install torch torchvision`)
- Additional packages: `pip install matplotlib scikit-learn`
- All previous week requirements (OpenCV, NumPy)

4.2 Recommended Datasets

- CIFAR-10: Natural images in 10 classes
- Fashion-MNIST: Clothing items classification
- MNIST: Handwritten digits (for quick testing)

4.3 Additional Reading

- PyTorch official tutorials: <https://pytorch.org/tutorials/>
- "Deep Learning with PyTorch" book (first few chapters)
- CNN architectures: LeNet, AlexNet basics
- Understanding overfitting and regularization

5 Next Week Preview

Week 4 will cover:

- Transfer learning with pre-trained models
- Advanced CNN architectures (ResNet, VGG basics)
- Real-world applications and datasets
- Model optimization and deployment basics

This week you'll build your first working deep learning models! Focus on understanding each component of the pipeline and don't worry if training takes time - that's normal.